CAIRN Implementation Specs

Rem

July 29, 2026

Abstract

CAIRN specifies task-agnostic metrics on an ECAN substrate: AF, STI/LTI, and Hebbian structure, sampled at CIP boundaries. Effectiveness is the ratio of aggregate metric delta to composite resource cost. This document gives the closed forms and MeTTa-style pseudocode independent of a particular language binding.

Contents

1 Background

1.1 Scope

Metrics are defined only on attention state. No external task score enters the formulas. Evaluation compares CIP snapshots and trajectories under a fixed agent/parameter regime.

1.2 ECANs

An ECAN can be represented as a graph consisting of the following atomic entities:

- Untyped nodes $A, B, C, \ldots$ representing concepts or knowledge items
- Untyped links between nodes $d, e, f, \ldots$ representing basic associations
- Typed links: HebbianLinks (H-links) and InverseHebbianLinks (IH-links)
- Weights: Short-Term Importance (STI) and Long-Term Importance (LTI)
- Assignment of STI/LTI weights to nodes
- Probability values assigned to typed links
- A forgetful operation to prune low-value atoms

STI is competitive attention mass (conserved modulo inflation and rent). LTI is retention value for keeping an atom in memory.

2 Core Metrics

2.1 Resource Cost Functions

Composite resource cost uses three normalized factors in $[0, 1]$:

- AF size ratio
- STI concentration
- AF-internal Hebbian link density

Listing 1: Ratio

```
(: measure-af-size-ratio (-> Number))
(= (measure-af-size-ratio)
    (let* (($af-size (count (get-af-members)))
           ($total-atoms (count (get-all-atoms-with-bins)))
           ($ratio (/ $af-size $total-atoms)))
        $ratio))
```

Listing 2: Concentration

```
(: measure-sti-concentration (-> Number))
(= (measure-sti-concentration)
    (let* (($af-members (get-af-members))
           ($af-sti (sum (map get-sti $af-members)))
           ($total-sti (sum (map get-sti (get-all-atoms))))
           ($concentration (/ $af-sti $total-sti)))
        $concentration))
```

Listing 3: Density

```
(: measure-link-density (-> Number))
(= (measure-link-density)
    (let* (($af (get-af-members))
           ($af-size (count $af))
           ($af-links (count-af-internal-hebbian $af))
           ($max-links (* $af-size (- $af-size 1)))
           ($density (if (> $max-links 0)
                         (/ $af-links $max-links)
                         0)))
        $density))
```

2.2 Effectiveness Calculation

Metric delta is the mean change in STI concentration and AF link density between two CIP snapshots. Resource cost is the sum of AF size ratio, STI concentration, and AF link density at the later snapshot. Effectiveness is their ratio (0 if cost is 0).

Primary reported effectiveness is **global** (baseline CIP → current). Local effectiveness (previous CIP → current) is recorded as a step series.

Listing 4: Attention Effectiveness

```
(: calculate-effectiveness (-> CIPSnapshot CIPSnapshot Number))
(= (calculate-effectiveness $from $to)
    (let* (($perf-delta (aggregate-metric-delta $from $to))
           ($resource-cost (total-resource-cost)))
        (/ $perf-delta $resource-cost)))

(: measure-local-effectiveness (-> CIPSnapshot CIPSnapshot Number))
(= (measure-local-effectiveness $prev $curr)
    (calculate-effectiveness $prev $curr))

(: measure-global-effectiveness (-> CIPSnapshot CIPSnapshot Number))
(= (measure-global-effectiveness $baseline $curr)
    (calculate-effectiveness $baseline $curr))
```

Listing 5: Redundancy

```
(: test-redundancy-degradation (-> CIPSnapshot CIPSnapshot (List Atom)
   PerformanceReport))
(= (test-redundancy-degradation $cip-prev $cip-curr $redundant-copies)
    (let* (($baseline-perf (aggregate-metric-delta $cip-prev $cip-curr))
           ($baseline-cost (total-resource-cost))
           ($_ (add-atoms-to-space $redundant-copies))
           ($degraded-cip (transition-to-next-cip $cip-curr))
           ($degraded-perf (aggregate-metric-delta $cip-prev $degraded-cip
              ))
           ($overhead (- (total-resource-cost) $baseline-cost))
           ($degradation-rate (/ (- $baseline-perf $degraded-perf)
              $overhead)))
        (PerformanceReport redundancy-impact $degradation-rate)))
```

3 Derived Metrics

3.1 Coherence and Retention

Listing 6: Coherence

```
(: measure-attention-coherence (-> Number))
(= (measure-attention-coherence)
    (measure-coherence (get-af-members)))
```

Listing 7: Retention

```
(: measure-context-retention (-> CIPSnapshot CIPSnapshot Number))
(= (measure-context-retention $prev-cip $curr-cip)
    (let* (($prev-af (get-af-snapshot $prev-cip))
           ($curr-af (get-af-snapshot $curr-cip))
           ($overlap (count-overlap
                       (map get-id $prev-af)
                       (map get-id $curr-af)))
           ($union-size (+ (count $prev-af)
                           (- (count $curr-af) $overlap))))
        (/ $overlap $union-size)))
```

3.2 Assessment

Listing 8: Assessment

```
;; Distributed importance: Pearson(mean STI series, current LTI)
;; over typeSpace / bank atoms
(: measure-distributed-importance (-> Number))
(= (measure-distributed-importance)
    (let* (($atoms (get-all-atoms))
           ($mean-stis (map
               (lambda ($a) (average (get-sti-series $a)))
               $atoms))
           ($lti-values (map get-lti $atoms)))
        (pearson-correlation $mean-stis $lti-values)))
```

Listing 9: Assessment

```
;; Selective modulation: Var(STI) / ((max-min)^2 / 12)
(: measure-selective-modulation (-> Number))
(= (measure-selective-modulation)
    (let* (($sti-values (map get-sti (get-all-atoms)))
           ($variance (variance $sti-values))
           ($span (- (maximum $sti-values) (minimum $sti-values)))
           ($null-var (/ (* $span $span) 12)))
        (if (> $null-var 0)
            (/ $variance $null-var)
            0)))
```

Listing 10: Assessment

```
;; Connection ratio: AF-out Hebbian with target in AF / all AF-out Hebbian
(: measure-connection-ratio (-> Number))
(= (measure-connection-ratio)
    (let* (($from-af (get-hebbian-links-from-af))
           ($total (count $from-af))
           ($internal (count-if
               (lambda ($l) (atom-in-af (get-target $l)))
               $from-af)))
        (if (> $total 0)
            (/ $internal $total)
            0)))
```

3.3 Audit

Listing 11: Audit

```
(: measure-preallocation-space (-> Number))
(= (measure-preallocation-space)
    (let* (($atoms (get-all-atoms))
           ($sti-values (map get-sti $atoms))
           ($min-sti (minimum $sti-values))
           ($shifted (map (lambda ($s) (- $s $min-sti)) $sti-values))
           ($total (sum $shifted))
           ($distribution (map (lambda ($s) (/ $s $total)) $shifted))
           ($entropy (shannon-entropy $distribution)))
        (/ $entropy (log2 (count $atoms)))))
```

3.4 Trajectory and Coordination

Listing 12: Trajectory

```
(: measure-cognitive-maintenance (-> (List CIPSnapshot) Number))
(= (measure-cognitive-maintenance $cip-series)
    (let* (($pairs (consecutive-pairs $cip-series))
           ($overlaps (map (lambda ($pair)
                             (measure-context-retention
                               (first $pair) (second $pair)))
                           $pairs)))
        (average $overlaps)))
```

Listing 13: Coordination

```
;; Glocal: geometric mean of local and cross-module coherence
;; Modules: weighted Louvain over all Hebbian links (weights = STV means)
(: measure-glocal-coordination (-> Number))
(= (measure-glocal-coordination)
    (let* (($hlinks (get-hebbian-links))
           ($modules (identify-modules $hlinks))
           ($local-coherence (average
                               (map measure-coherence $modules)))
           ($inter-links (inter-module-links $hlinks $modules))
           ($link-probs (map get-probability $inter-links))
           ($conjunctions (map
               (lambda ($link)
                   (temporal-conjunction
                       (get-source $link)
                       (get-target $link)))
               $inter-links))
           ($global-coherence
               (pearson-correlation $link-probs $conjunctions)))
        (sqrt (* $local-coherence $global-coherence))))
```

3.5 Benchmark

Listing 14: Gained Efficiency

```
(: measure-gained-efficiency (-> CIPSnapshot CIPSnapshot
                                 CIPSnapshot CIPSnapshot Number))
(= (measure-gained-efficiency $base-prev $base-curr $opt-prev $opt-curr)
    (let* (($baseline-eff (calculate-effectiveness $base-prev $base-curr))
           ($optimized-eff (calculate-effectiveness $opt-prev $opt-curr)))
        (/ (- $optimized-eff $baseline-eff) $baseline-eff)))
```

4 Instrumentation

4.1 Time Series

Listing 15: STI/LTI Time Series

```
;; Global partial maps from atom-id to list of values
;; one entry appended per CIP transition
(= sti-series (new-state (empty-map)))
(= lti-series (new-state (empty-map)))

;; Called at each CIP boundary before AtomSpace is updated
(: record-cip-series (-> ()))
(= (record-cip-series)
    (map (lambda ($atom)
             (let* (($id (get-id $atom))
                    ($curr-sti (get-sti $atom))
                    ($curr-lti (get-lti $atom))
                    ($prev-sti (map-get (get-state sti-series) $id))
                    ($prev-lti (map-get (get-state lti-series) $id)))
                (set-state sti-series
                    (map-put (get-state sti-series) $id
                             (append $prev-sti $curr-sti)))
                (set-state lti-series
                    (map-put (get-state lti-series) $id
                             (append $prev-lti $curr-lti)))))
         (get-all-atoms)))

(: get-sti-series (-> Atom (List Number)))
(= (get-sti-series $atom)
    (map-get (get-state sti-series) (get-id $atom)))

(: get-lti-series (-> Atom (List Number)))
(= (get-lti-series $atom)
    (map-get (get-state lti-series) (get-id $atom)))
```

4.2 Topological Analysis

Listing 16: Persistent Homology

```
;; Clique-complex invariants (triangles + Betti 0/1/2 via mod-2 ranks)
;; Incremental over Hebbian edge additions between CIPs
(: persistent-homology-invariants (-> TopologyReport))
(= (persistent-homology-invariants)
    (let* (($edges (hlinks-to-edge-list))
           ($report (topology-invariants $edges)))
        (TopologyReport
            (topology-triangles $report)
            (topology-betti-0 $report)
            (topology-betti-1 $report)
            (topology-betti-2 $report))))
```

4.3 Utilities

Listing 17: Coherence helpers

```
;; Time-averaged STI co-activation of two atoms over a CIP trajectory
(: temporal-conjunction (-> Atom Atom Number))
(= (temporal-conjunction $atom-i $atom-j)
    (let* (($sti-i (get-sti-series $atom-i))
           ($sti-j (get-sti-series $atom-j)))
        (average (map * $sti-i $sti-j))))

;; Coherence: Pearson of link probabilities vs temporal conjunctions
(: measure-coherence (-> (List Atom) Number))
(= (measure-coherence $atoms)
    (let* (($hlinks (get-hebbian-links-within $atoms))
           ($link-probs (map get-probability $hlinks))
           ($conjunctions (map
               (lambda ($link)
                   (temporal-conjunction
                       (get-source $link)
                       (get-target $link)))
               $hlinks)))
        (pearson-correlation $link-probs $conjunctions)))
```

4.4 CIP Records

Listing 18: CIP Tracking

```
(: CIPSnapshot Type)
(: CIPSnapshot (->
    Number          ; CIP index (0 = baseline)
    Number          ; timestamp
    (List Atom)     ; AF membership frozen at snapshot time
    (List Metric)   ; all metrics measured at this boundary
    CIPSnapshot))

(: initialize-baseline-cip (-> CIPSnapshot))
(= (initialize-baseline-cip)
    (let* (($_ (record-cip-series))
           ($af (get-af-members))
           ($metrics (measure-all-metrics)))
        (CIPSnapshot 0 (current-time) $af $metrics)))

(: transition-to-next-cip (-> CIPSnapshot CIPSnapshot))
(= (transition-to-next-cip $prev-cip)
    (let* (($_ (record-cip-series))
           ($new-index (+ (get-cip-index $prev-cip) 1))
           ($af (get-af-members))
           ($metrics (measure-all-metrics)))
        (CIPSnapshot $new-index (current-time) $af $metrics)))
```

4.5 Attention Trend Analysis

Listing 19: STI Trend and Volatility

```
;; STI trend for a single atom: direction of attention change at run end.
;; Compares the most recent STI value to the series mean.
(: sti-trend (-> Atom Symbol))
(= (sti-trend $atom)
    (let* (($series (get-sti-series $atom))
           ($latest (last $series))
           ($avg    (average $series)))
        (if (> $latest $avg) rising
            (if (< $latest $avg) falling
                stable))))

;; STI volatility: standard deviation of the atom's STI series.
;; Near 0 = stable attention across CIPs; higher = fluctuating.
(: sti-volatility (-> Atom Number))
(= (sti-volatility $atom)
    (standard-deviation (get-sti-series $atom)))
```

At end of each run trends are written to `trends.csv` (columns: `atom`, `trend`, `volatility`) for direct visualization, and included in `summary.json` under `atom_trends`. Three aggregate metrics derived from the per-atom data are added to `summary.json` under `af_trend_summary`:

Listing 20: Derived Attention Trend Metrics

```
;; Fraction of AF atoms with stable STI trend at run end.
;; Near 1 = attention fully settled; near 0 = all atoms in flux.
(: af-stability-ratio (-> Number))
(= (af-stability-ratio)
    (/ (count-if (== trend stable) (af-trend-entries))
       (length (af-trend-entries))))

;; Mean STI volatility across AF atoms.
;; Near 0 = low cross-CIP STI variance; higher = high turbulence.
(: mean-af-volatility (-> Number))
(= (mean-af-volatility)
    (average (map volatility (af-trend-entries))))

;; Net attention flow at run end: rising minus falling atoms.
;; Positive = more atoms gaining salience; negative = more losing it.
(: attention-trajectory (-> Number))
(= (attention-trajectory)
    (- (count-if (== trend rising)  (af-trend-entries))
       (count-if (== trend falling) (af-trend-entries))))
```

References

[1] Iklé, M., Pitt, J., Goertzel, B., & Sellman, G. (2010). *Economic Attention Networks: Associative Memory and Resource Allocation for General Intelligence*. Proceedings of the Third Conference on Artificial General Intelligence.

[2] Coward, L. A. (2001). *The Recommendation Architecture: Lessons from Large-Scale Electronic Systems Applied to Cognition*. Cognitive Systems Research, 2(2), 111-156.

[3] Sellman, G. & Goertzel, B. (2011). *Using Economic Attention Networks for Controlling Embodied Intelligent Systems.*

[4] Goertzel, B. et al. *Using Nonlinear Dynamical Attention Allocation to Focus Probabilistic Logical Inference Upon Relevant Information.*